

No Silver bullet

Resumen y Accidents en lo Ing. De SW

No existe una Solución mágica

No existe un solo avance → Que mezcle la productividad
Confabilidad o Simplicidad
(En tecnología o técnicas de gestión)

Tareas esenciales vs. Accidentales

Creación y modelado
de estructuras conceptuales
complejas

Representación de estos
entidades abstractas en Leng. De programación
Su adaptación al lenguaje de la máquina bajo
restricciones de espacio y velocidad

Las mejoras → Vienen a reducir la
historias accidental pero no al 100%

Problema central → Lo esencial

Estrategia
clave
para atacar
al problema

Reutilizar SW → Comprar SW

Usar prototipos → Para definir los requisitos

Desarrollar de forma iterativa → Añadir funciones mientras se
ejecuta, uso y prueba.

Apostar por buenos
diseñadores

Los proyectos pueden
Volverse caóticos

→ No existe Solución
mágica.

¿Tiene que ser difícil? Las dificultades escolares del SW

La Esencia del SW

La mayor dificultad
en la construcción
del SW

→ esto en la → especificación { De la
Diseño
Prueba

No está en la
Codificación
Representación

Propiedades que lo hacen inevitablemente Complejo

Complejidad

Muchos elementos distintos
con interacciones no lineales

→ Desencadena en problemas de comunicación,
defectos en el producto, exceso de costos,
retrazos y desafíos de gestión

Conformidad

Se debe adoptar el
producto a múltiples interfaces
impuestos por

→ Instituciones
u otros sistemas
preexistentes

→ El SW es lo más
nuevo y más adaptable.

Cambiability (Mutabilidad)

El SW está en
constante modificación

Invisibilidad

No tiene forma
física

→ Difícil de entender
y de comunicarlo.

Los avances del pasado resolvieron dificultades accidentales

Lenguajes de alto nivel

Aumentaron la Productividad en el software
Confiableza
Simplificación

Programadores se enfocaron ideas y no en bits

Si se vuelven muy complejo dejan de ser útiles.

Time-Sharing

Elimina los tiempos de espera → Mayor fluidez y productividad

Cuando la respuesta es inmediata → No hay mejoras adicionales relevantes.

Entornos de programación unificados

Los programas trabajan colaborativamente

Permite reutilización y combinación de herramientas

Esperanza tecnológica para una bola de plata

ADA/Lenguajes de alto nivel

Fomenta la modularidad y el diseño moderno

Los mayores beneficios se obtuvieron en el pasado

Programación orientada a objetos

Mejora la estructura y construcción del SW

Elimina dificultades accidentales en la expresión del diseño

Inteligencia artificial

Poco impacto en la programación → No es escribir código sino definir ¿Qué hacer?

Swas expertos

Puede ayudar en la
destrucción

Difícil capturar la experiencia
y sabiduría de los mejores programadores

Programación "Automática"

Útil en
Casos específicos
y bien parametrizados

Imposible de generalizar
en el desarrollo de sw ordinario
y complejo.

Programación Visual

Poco útil → El sw no tiene forma
visual clara.

Verificación de programas

Solo valida que
el sw cumple con
la especificación

Lo más difícil es crear y definir
una especificación completa y coherente

Entornos de desarrollo y estaciones de trabajo

Seguirán mejorando
pero con rendimientos
decrecientes

→ No mejoran la productividad
del programador que pasa el
tiempo pensando.

Estrategias prometedoras para abordar la esencia del sw

Ataques a la esencia Conceptual

"Mejorar" la
expresión
herramientas, lenguajes

→ No alcanza
(No mejoran la productividad)

Comprar en lugar de construir

Usar sw existente es
más barato y eficiente

→ Que crearlo de
0

El HW es más
accesible

→ Las empresas hoy
adoptan procesos
al sw.

Herramientas como
hojas de cálculo y
Bases de Datos

→ Permiten resolver problemas
a los trabajadores.

Refinamiento de requisitos y prototipos rápidos

Parte más difícil → Que construir

Los clientes no saben lo que quieren hasta que ven el Sma en acción → Es imposible que los clientes especifiquen los requisitos de forma completa y correcta

Solución → Prototipos rápidos + Desarrollo iterativo

Desarrollo Incremental (Cultivar en lugar de construir)

El SW no debe construirse → Debe cultivarse

El desarrollo debe ser incremental (De arriba hacia abajo) → El programa/Sma debe ejecutarse aunque llame a subprogramas vacíos
↓
Luego se añade funcionalidad

SW que funciona en el momento → eleva la moral del equipo
↓
SW crece orgánicamente adaptándose a circunstancias complejas → Logrando Smas robustos

Identificar y nutrir grandes diseñadores

El núcleo para mejorar el arte del SW → radica en las personas

Hay diferencias entre diseñadores → Esto en el talento

Los mejores diseñadores producen resultados superiores

Hay que identificar, formar y valorarlos

Resumen

No existe una
bola de plata → Ningún avance va a
revolucionar el SW
Como ocurriría con el HW

Progreso será
gradual → Requiere trabajo constante,
disciplinado y acumulativo

La clave está
en la esencia del
problema → Enfoca en la comprensión
conceptual
↳ No solo en la herramienta
o tecnología.

Estrategias reales
a largo plazo → Reutilizar SW existente
↳ Usar prototipos e iteración
↳ Desarrollar de forma gradual
↳ Apoyar por grandes desarrollos